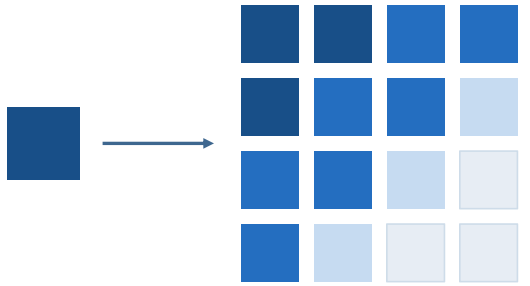
技术分享

从单点加速到系统最优
大模型推理的全栈优化实践



以混元大模型推理优化实践为例：从底层算子到推理框架的全栈协同

目标与瓶颈 → 算子级优化 → 系统级协同 → 验证与成果

混元 AI Infra

Tencent 腾讯

分享主线：以服务目标为导向的全栈优化

以「算 / 搬 / 存 / 等」四类瓶颈为线索：算子让每一步更快，系统协同解决容量、分工与调度，最后验证总体成效

01 目标与瓶颈分析

核心问题

用户为什么会等?

- **一次请求的 Prefill / Decode**
  首字时延与逐字时延从哪来
- **算 / 搬 / 存 / 等四类压力**
  计算、访存、显存容量、排队
- **全栈体系总览**
  四类瓶颈对应的优化层次

02 算子级优化

核心问题

单点计算能做到多快?

- **HPC-Ops 生产级算子库**
  Attention、MoE 等核心计算
- **按算 / 搬 / 等定位浪费**
  任务均衡、数值重构、索引直读
- **融合执行与通算协同**
  压缩访存、启动与跨卡交接

03 系统级协同

核心问题

每一步都更快了，系统为何仍受限?

- **为什么需要系统级协同**
  四类问题的系统解法
- **量化方式**
  低比特表示与精度保持
- **KV 归属、流水与冷热分级**
  Prefill / Decode 的显存与并行
- **并行投机解码**
  把时延余量变成有效产出
- **全局调度与 KV 池化**
  拥塞感知，减少排队等待

04 验证与成果

核心问题

收益如何被证明?
总体达成了什么?

- **Lyra 仿真验证**
  生产同构的流量回放
- **总体成效**
  首字、逐字、承载与质量
- **四个认知与优化闭环**
  目标、瓶颈、资源与系统

目标与瓶颈分析 — 算子级优化 — 系统级协同 — 验证与成果：把局部加速转化为可验证的系统收益

案例均来自混元大模型推理优化实践

Tencent 腾讯

01

PART 01 / 04

目标与瓶颈分析

用户为什么会等？瓶颈真正在哪里？

本章要点

1 一次请求的 Prefill / Decode：用户在等什么

2 算 / 搬 / 存 / 等：四类瓶颈压力

3 全栈优化体系总览：瓶颈与优化层次的对应

Tencent 腾讯

从一次请求看推理瓶颈：算 / 搬 / 存 / 等

上下文越长、请求越多，一次请求经过的每个阶段都在放大计算、搬运、容量与等待四类压力

一次请求：先读完上下文，再逐 Token 生成

用户感知：排队 → 首字 → 逐字



算得多 · 计算

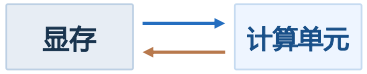


表现 长上下文使 Prefill 计算负担重

例 MoE、Attention 等核心计算：主流算子库在主力卡 H20 上性能释放不足

→ 02 算子 → 03 系统

搬得多 · 访存 · 通信



权重 / 中间结果反复读写

表现 权重与中间结果反复进出显存

例 多卡 AllReduce 归约与通信；十余个采样 Kernel 的启动开销

→ 02 算子 → 03 系统

存不下 · 显存容量



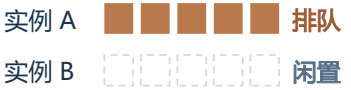
随长度增长，可放下的请求变少

表现 长上下文使 KV 缓存快速增长，挤压并发空间

例 Hy4 长文本 Decode 中，KV 已成最主要的显存瓶颈；而权重占用了大量显存

→ 03 系统

等得久 · 负载 · 同步



表现 稀疏专家与动态请求造成负载倾斜

例 Session 亲和：局部排队与全局闲置并存；EP 专家冷热不均，rank 间同步空转

→ 03 系统

服务目标

先过门槛，再比产出

门槛

质量不降

提速不能以输出质量为代价

时延达标 TTFT TPOT

例：单 token 输出时延满足 SLO 目标

首字与逐字都要满足 SLO

在此之上

目标

提高单位资源的达标产出

四类压力既会侵蚀门槛，也会压低达标产出

难点不是单纯“算快一点”：还要兼顾搬运、容量与等待，且不牺牲质量

四类压力为归纳框架；后续每一章对应其中一类或几类

全栈推理优化体系总览

服务目标　**长上下文、高并发、低时延与模型质量需同时满足**
在时延与质量约束下提升单位资源服务产出

业务场景　**语言模型在线服务 / 多模态生成服务**
智能体（Agent） / 编程（Coding） / 多轮对话 / 图像生成 / 3D 生成

全栈推理优化体系

模型执行、集群服务与硬件部署协同

集群服务层

降低排队等待
提高集群产出

服务系统　全局负载调度、缓存共享与服务编排　03 · 等　04 · 验证

- **LWL 全局调度器**
  工作量调度 / 拥塞感知
- 缓存感知
- **KV 共享缓存池**
  共享前缀 / 按需拉取
- **Lyra 推理服务仿真器**
  流量回放 / 性能预测 / 策略寻优
- 策略验证与反馈
- **多模态服务编排**　阶段调度 / 状态管理 / 异常恢复

模型执行层

扩大并发空间
提高生成效率

推理框架

- **混元推理框架**　03 · 存 / 分
  并行布局 / KV 分层 / DFly 并行投机解码 / 异步调度
- 计算支撑
- 压缩模型

核心算子　02 · 算 / 搬

- **HPC-Ops 生产级算子库**
  高效计算 / 算子融合 / 通算协同

量化压缩　03 · 存

- **AngelSlim 模型压缩工具包**
  低比特量化 / 精度补偿 / 部署适配

硬件部署层

复用公共能力
拓展硬件供给

异构适配　部署形态（本次不展开）

- **统一平台抽象与适配**
  专用算子与通信后端 / 分布式部署
- **语言模型推理**
- **多模态生成**
- 异构芯片

计算、访存、通信与调度跨层协同，把局部加速转化为端到端服务收益

02

PART 02 / 04

算子级优化：先把“算”和“搬”做到极致

在主力推理硬件上，单点计算能做到多快?

本章要点

1. HPC-Ops 生产级推理算子库
2. 按算 / 搬 / 等定位每类算子的浪费
3. 融合执行与通算协同

Tencent 腾讯

HPC-Ops：生产级推理算子库

算：在主力推理硬件上把核心计算做到极致——贯通 Attention、MoE、GEMM、通信、采样与前后处理

业务服务 线上推理服务 — 智能体 / 编码 / 长上下文服务

↓

推理框架 vLLM / SGLang（Python / PyTorch API）— 原生后端接入，覆盖预填充与解码；动态批次适配 / 图执行 / 工作区复用

↓

HPC-Ops 核心计算、数据处理、融合执行与通信协同

Attention 最高加速 2.95×
动态均衡与跨层调度复用
GQA / MLA / Sparse MLA
BSA / SageAttention

GEMM Router 最高 3.22×
高精度重构与门控融合
Dense / Grouped / RouterGEMM
GatedMLA

MoE Grouped GEMM 最高 1.88×
索引直读与专家融合流水
FusedMoE / MoE-EP
Quant / Shared Expert

Indexer / TopK
分页评分与精确稀疏选择
MQA Indexer
TopK / GroupedTopK

Sampler
单次词表读取与设备状态闭环
FusedSampler

Quantization
多粒度量化与计算布局重排
Quant / Dequant
Hadamard

Transform / Fusion
跨阶段融合，压缩中间读写
iHC / mHC
RoPE / Norm / Act

Communication
通信融合与分片通算重叠
AllReduce / GEMM-RS
QComm

KV / State
KV 压缩与递推状态更新
KVCompressor
Mamba

↓

数值格式 FP32 / BF16 / FP8 / INT8 / INT4
量化配置 W4A4 / W4A8 / W8A8 / W8A16 — 格式与配置按算子提供

规模化生产成效

混元生产 QPM 提升
30%
DeepSeek 提升 17%

逐字时延最高降低
48.8%

大规模生产集群
全量上线
混元 / DeepSeek 等模型

生态共建
vLLM / SGLang
核心算子作为原生后端
合入社区主线

取舍：自研换性能与可控性，需持续多格式、多 shape 适配与回归 | 适用边界：收益依赖目标硬件与模型结构，经统一接口接入框架才能兑现

算子级优化：先消除每一步的浪费

依托 HPC-Ops 的生产级能力，沿「算 / 搬 / 等」定位每类算子的浪费，再为它选择合适的机制

共同背景 **设备并不总在做有用计算**：长短任务拖尾、精度敏感计算、反复搬运与阶段等待，会在每一层、每一步反复出现

算子	浪费在哪（算 / 搬 / 等）	机制：如何减少浪费	结果（各自基线）
Attention GQA / Sparse MLA	等 长短请求混排，任务拖尾	动态任务均衡、持久化执行与跨层调度复用；贯通分页 Indexer 评分与精确 Top-K	**最高 2.95×** Attention 算子
Router GEMM MoE 路由	算 精度敏感，不能简单降精度	以双 BF16 重构高精度 Router，兼顾数值精度与 Tensor Core 效率	**最高 3.22×** Router 算子
MoE 专家计算 Grouped GEMM / FusedMoE	搬 等 多阶段串联，Gather 与布局开销	索引直读、紧凑布局与流水调度；整合 Top-K、量化、分发、计算与聚合	**最高 1.88×** Grouped GEMM
算子链融合 前后处理 · 采样	搬 中间态反复进出显存，启动多	iHC 前后处理、RoPE / Norm / Hadamard / 量化 / KV 写入融合；十余个采样 Kernel 压成两个	**超 9× / 约 5.5×** iHC / Sampler
通算融合 多卡归约 · 通信	搬 等 跨卡交接多，计算与通信串行	AllReduce + 残差 + RMSNorm 融合；GEMM—ReduceScatter 分块流水，就绪即通信	**1.68× / 1.81×** 归约融合 / GEMM–RS

先回答浪费发生在哪里，再选择机制：同样是“快”，不同算子要解决的问题并不相同

各项为对应基线下的最高加速比，不相加、不相乘，也不等同于端到端服务加速；融合与通算细节见下一页

HPC-Ops：融合执行与通算协同

搬 / 等：减少中间态往返、启动与跨卡交接

下一问：每一步都更快了，系统就不再受限了吗?

跨阶段融合与片上处理

跨阶段数据复用
iHC Post–Pre

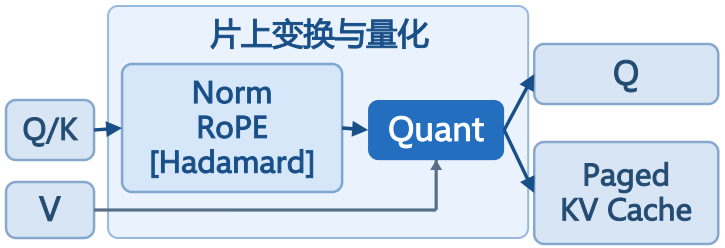
超 9× 对 Torch



跨段共享统计，压缩归约与启动

注意力后处理融合
QKV Postprocess

约 5× 对拆分路径

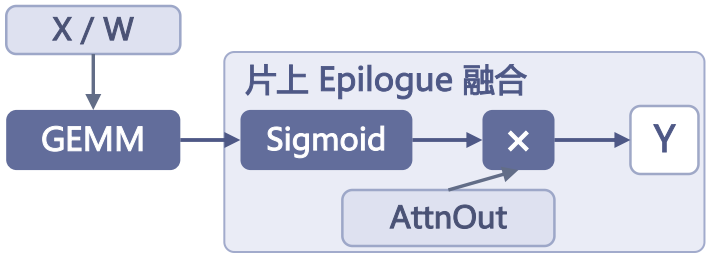


中间结果片上流转，直接写入 KV

门控融合与采样加速

门控投影融合
Gated MLA

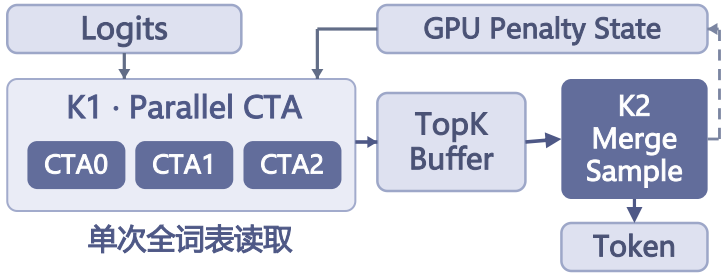
1.30× 对 Torch



门控随 GEMM 完成，省去中间写回

采样流程融合
FusedSampler

约 5.5× 对 vLLM

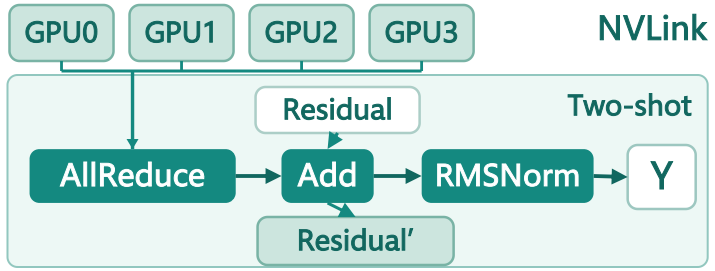


词表单次扫描，采样状态留在 GPU

通信融合与通算协同

归约与后处理融合
AllReduce + RMSNorm

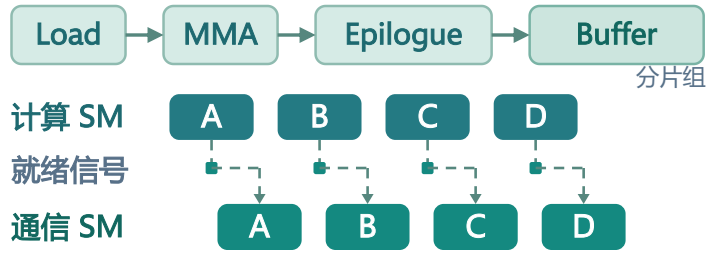
最高 1.68× 对开源路径



归约结果就地处理，压缩独立后处理

计算通信分片重叠
GEMM + ReduceScatter

1.81× 对串行路径



就绪即通信，与后续计算重叠

取舍：融合压缩访存与启动开销，但提高算子复杂度 | 适用边界：访存 / 启动受限的前后处理与多卡归约场景

基线：归约融合对 NCCL / FlashInfer；GEMM–RS 对 GEMM 与 ReduceScatter 串行执行

03

PART 03 / 04

系统级协同：容量 · 分工 · 调度

每一步都更快了，系统为什么仍会受限?

本章要点

1. 为什么需要系统级协同：四类问题的系统解法
2. 量化方式：压缩表示，腾出显存空间
3. Prefill / Decode：KV 归属、流水与冷热分级
4. 并行投机解码：把时延余量变成有效产出
5. 全局调度：KV 池化与拥塞感知

系统级协同：把局部效率变成服务能力

算子让每一步更快，却不能单独解决「算 / 搬 / 存 / 等」：容量、分工与调度，需要框架与系统协同

背景：每一步都更快了，系统仍可能被**容量、分工和调度**限制——腾出的资源，要靠框架组织起来才能承载更多请求

问题	02 算子级已经做到	仍未解决：系统级约束	03 本章方法（页码）
算得多	核心计算与融合让单步更快	**Decode 逐 Token 串行：**每步只产出一个 token，单步再快也要走完整条序列	**并行投机解码：**一次校验接受多个 token p15
搬得多	融合减少中间态往返，通算分块流水	**跨卡冗余与通信：**CP 下每个 rank 复制全部层 KV；EP 下 token 集合通信随规模放大	**KV 按层划分归属；**专家权重拉取档位随长度切换 p13
存不下	无法改变权重与 KV 的容量需求	**显存先满：**Decode 单步耗时远低于 SLO 目标，瓶颈在显存；长上下文 KV 挤压并发	**低比特量化；**流水释放权重显存 + KV 冷热分级 p12 p14
等得久	无法决定请求去哪、何时执行	**排队与空等：**会话绑定单实例，局部排队与全局闲置并存；CPU 调度留下 GPU 空泡	**LWL 全局调度 + KV 池化；**异步调度压缩空泡 p16 p15

本章主线： 先腾出容量 → 再组织分工 → 用时延余量换产出 → 减少排队等待

每一步都更快了，还需要框架与系统协同，才能把局部效率变成服务能力

按主要问题归类，不代表每项优化只解决一种瓶颈；各案例基线独立，收益不累计

量化方式：压缩表示，为 KV 与并发腾出空间

存：权重与激活以低比特表示，降低显存占用与访存带宽；难点是离群值放大量化误差

多层次量化体系：权重 → 激活 → Attention → 整体

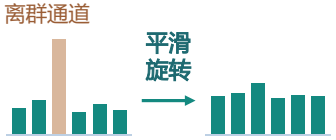
瓶颈：权重低比特误差 × 激活离群值

权重侧 · GPTQ



逐层重建补偿权重量化误差，显著降低精度损失

激活侧 · 平滑 + 旋转



逐通道平滑因子压缩离群值且不改变输出；旋转矩阵稳定分布

Attention 量化



在线正交旋转 · 离群值打散

Q/K 量化前在线正交旋转；计算高效、可在线融合，几乎不增开销

整体 · 轻量 QAT



量化参数初值附近微调适配任务分布；仅更新部分参数，收敛快

联合优化 → 4 比特量化：多领域评测与 BF16 基线持平或差距在 1% 以内

MTP 续训：低成本注入自推测能力



草稿分布与推理分布对齐，严格保证输出分布不变；续训成本可控

压缩 → 算子 → 部署

AngelSlim → HPC-Ops → 推理部署

格式 FP8/INT8/INT4

配置 W4A4/W4A8/W8A8/W8A16

格式与配置按算子提供，压缩收益在部署端兑现

验证 · Hy 模型 4 比特

多领域评测集 · 对 BF16 基线

精度差距

≤ 1%

持平或差距在 1% 以内

MTP 续训

续训 token 量较原始预训练

低两个数量级

以上 · 推理引擎侧改动最小

量化覆盖
8bit · 4bit · 极低比特
Attention 量化

取舍：压缩比 vs 精度，校准与 QAT 引入额外离线成本

适用边界：4bit 需配合离群值处理与 QAT 才能精度无损；需硬件与算子支持低比特格式，才能兑现端到端收益

长上下文 Prefill：KV 归属与并行档位

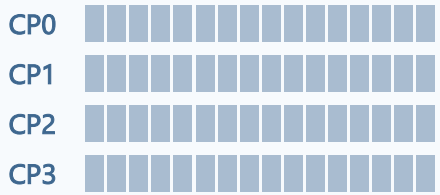
存 / 搬：KV 不再每卡复制，按层划分归属；专家权重拉取档位随长度切换，收缩 token 通信

CP KV LayerSplit · 按层切分 KV 所有权

机制示意

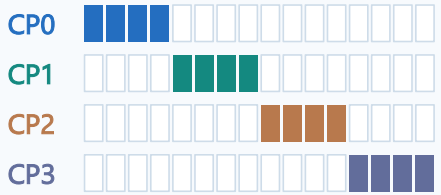
CP 下同一长序列切分到各 rank 并行预填充，但每层 KV 在每个 rank 上完整复制一份，显存随 CP 度放大

复制：每个 rank 常驻全部层



按层切分

LayerSplit：只常驻 owner 层



示意 4 个 rank × 16 层；彩色为 owner 层，白格为非常驻层（用时由 owner 广播）

写路径 当前 token 各 rank 本地写，零通信

读路径 owner 只广播 context 页
non-owner 按需 scatter 取用

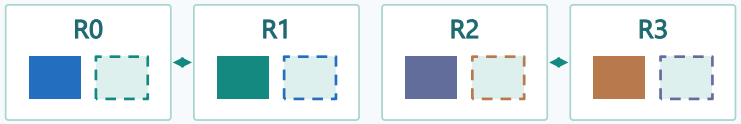
每 rank 常驻 **1/8 层 KV**

KV 容量 **≈4.43×**
TTFT **3.8～4.7×**

DW · 专家权重拉取：EP 度 8 → 4 / 2 / 1

把邻近 rank 的专家权重经 NVLink PULL 到本地，等效扩大单 rank 专家持有量；token 集合通信随 EP 度收缩

DW2 示意：成对 rank 互相 PULL 对方专家（虚线框）



- **降通信**：token AG/RS 随 EP 度收缩，DW8 时消失，换成与 batch 无关的固定权重搬运
- **降不均衡**：可见专家变多，缓解冷热不均长尾

序列长度 → 最优档位

纯 EP · ≤8K	DW2 · 16K 起反超 7%–13%	DW4 · ≥128K

DW8：每层固定权重搬运，各长度约 1.7s 底板，始终不占优

65536	DW2 999ms	vs	纯 EP 1136ms
196608	DW4 3183ms	vs	纯 EP 3461ms

长上下文 Prefill

CP KV LayerSplit · DW

KV 容量提升约

4.43×

Prefill TTFT 加速

3.8～4.7×

上下文越长，收益越大

DW2 vs 纯 EP（≥16K）

7%–13%

DW4 在 128K 以上接棒最优

取舍：用 owner 广播与权重搬运，换取 KV 显存容量与 token 通信收缩

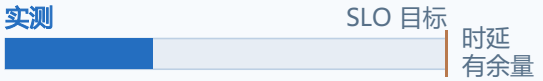
适用边界：上下文越长收益越大；DW 档位随序列长度切换，8K 以内纯 EP 最优，DW8 搬运开销过大

Tencent 腾讯
03 系统级协同

长上下文 Decode：释放权重显存 + KV 冷热分级

存 / 分：时延有余量、显存是瓶颈——流水线释放权重显存，KV 按冷热分级卸载到主机内存

瓶颈判断 单步解码耗时远低于 SLO 时延目标
→ 瓶颈是**显存容量**而非计算时延



Decode PP3 · 三级流水

DP8EP8 上再按层拆 3 段；沿批次切
3 个独立子批次交错推进，填满流水



权重显存（PP3 按层三分）



KV 可用容量 **+53GB**

KV Cache Offloading · 热 / 冷分层

机制示意

D 节点 KV 已 FP8，再压 FP4 精度风险大 → 精度无损地用搬运换容量

前提1 访问稀疏	前提2 访问局部性	前提3 内存闲置
每层只取 top-k=2048	200 步仅选中约 4k ~ 12k	D 节点 CPU 内存基本空置

GPU 显存 · 热 KV（indexer KV 常驻）

Indexer（每层选 top-k=2048） → GPU Slot Pool（token 粒度常驻热 KV） → Sparse MLA（+ UVA Sync Load 兜底）

PCIe · CPU 制定 migration plan · CUDA Graph 全捕获

CPU 内存 · 冷 KV（显存:内存 ≈ 1:1） H2D 预取+按需 D2H LRU 换出

- CPU Slot Pool：冷 KV 存放 / 换出目标
- ResidencyDirectory：logical token → GPU / CPU / 未分配

L0 miss 率 **1% ~ 2.29%**
L1 ~ L3 miss 率 **< 0.3%**

step0 命中 **97.8% ~ 99.4%**
P 侧按 token 热度重排；稳态 99.4% 以上

Hy4 长上下文 Decode

256K 纯 Decode 测试

Decode 并发

×2

并发容量翻倍

吞吐提升

44%

PP3 KV 可用容量

+53GB

权重显存 80GB → ~26.67GB
KV 精度保持 FP8 不变

取舍：用 PCIe 搬运与流水气泡，换取 Decode 显存容量与并发

适用边界：依赖稀疏注意力的访问稀疏性与局部性；Decode 受显存而非算力约束；需有闲置主机内存

并行投机解码：把时延余量变成有效产出

算 / 等：逐 Token 串行、每步只出一个 token——用一次校验接受多个 token，并异步压缩 GPU 空泡

瓶颈：草稿时延与接受长度难以兼得

自回归草稿
MTP · 逐 token



4 次前向
时延随长度线性增长

并行块草稿
DFlash · 双向注意力



1 次前向
时延几乎与块长无关

■ 共享上下文限制层差异化建模、位置间缺乏自回归依赖 → 深层接受率偏低

平均接受长度 · Hy3-A21B

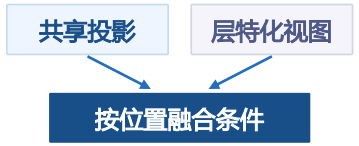


+59.7% 对 MTP　+29.8% 对 DFlash

DFly：从草稿建模到部署调度的四项协同设计

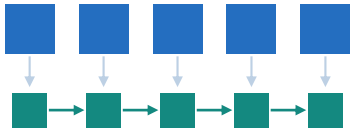
六项数学 / 代码 / 对话基准全面领先

① 混合条件骨干



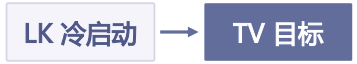
DFlash 共享投影 + DFlare 层特化视图按位置融合；少量标量权重可预计算

② 自回归校正头



骨干保持全并行，仅小型序列头自左向右，补足块内自回归依赖

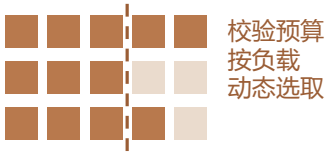
③ 接受度对齐训练



目标 = 期望接受长度

D-PACE 混合 LK 损失冷启动 → 端到端 TV 目标，直接优化期望接受长度

④ D-cut 动态校验



按逐位置置信度与单步时延选最优校验预算，高置信前缀打包为稠密批

框架层　CUDA Graph　算子融合　Chunk Prefill　差异化 KV 管理　一致量化　多卡权重共享　异步调度压空泡

验证 · Hy3-A21B

相对自回归解码加速

1.98 ~ 2.40×

较 DFlash 吞吐再提升

10.5% ~ 11.8%

生产流量 · D-cut

+15.7%

并发饱和后再提升（最高）

拓宽有效并发区间
输出分布严格保持不变

取舍：草稿越长接受越多但校验时延越高，需按负载动态选择校验预算

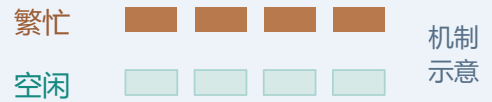
适用边界：代码/数学等可预测跨度长的场景收益更大；高并发饱和后收益收窄，需 D-cut 补位；输出分布严格不变

全局调度：KV 池化与拥塞感知

等：容量腾出来之后，请求仍会在忙实例上排队——以 KV 池化解耦缓存归属，以拥塞感知选择实例

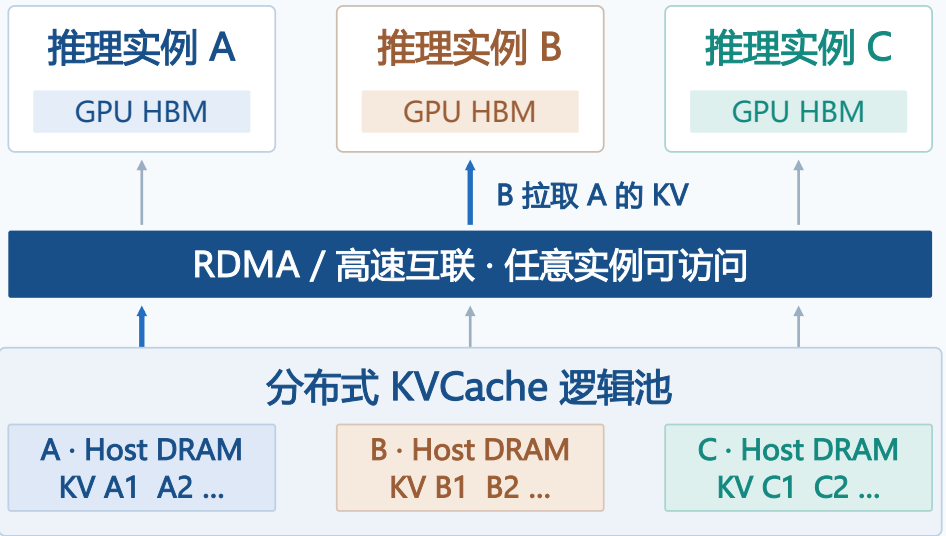
纯 Session 亲和：局部排队与全局闲置并存

缓存绑定单实例，空闲算力难以承接热点请求



KVCache 池化 · 跨实例复用

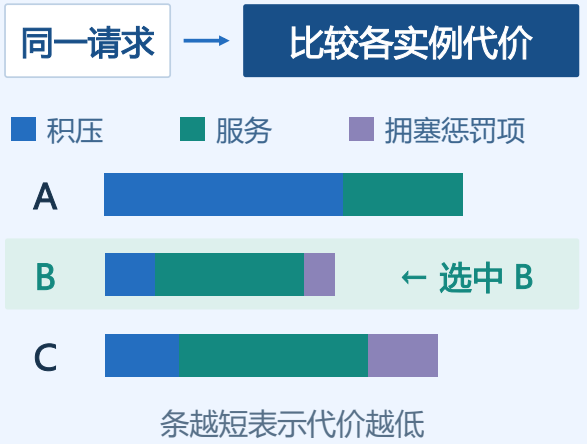
缓存分布式存放，前缀按需跨节点访问



无需全量重计算；箭头表示 KV 拉取，蓝色代表跨节点拉取

LWL · 拥塞感知选点

Least Work Left Scheduler



条越短表示代价越低

积压工作量 + 服务成本
+ 拥塞价格 × 额外服务成本

低拥塞促分流；高拥塞抑制额外开销

Hy3 上线压测

固定并发 · 对比 Session 亲和

TTFT 降低 **48.1%**

TPS 提升 **39.6%**

Hy3 正式版落地

TPS 基本持平 · 对比 Session 亲和

平均 TTFT 降低 **12.9%**

P99 TTFT 降低 **27.3%**

与业界SOTA方法对比

Qwen · 24P8D · 对 SMetric

平均 TTFT 降低 18.1%

平均排队时延降低 58.8%

TPS 基本持平

取舍：在缓存复用收益与排队代价之间定价——跨实例拉取 KV / 重计算 vs 等待繁忙实例

适用边界：负载不均、拥塞明显时收益最大；依赖 KV 池化与 RDMA 高速互联

04

PART 04 / 04

验证与成果

收益如何被证明？总体达成了什么效果？

本章要点

1. Lyra：生产同构的仿真验证
2. 总体成效：多项优化协同后的服务能力
3. 四个认知与优化闭环

Lyra：生产同构的仿真验证

构建 vLLM 兼容的推理模拟服务，以纯CPU环境支撑真实调度链路的回放与寻优

生产链路直接接入，保留真实调度策略

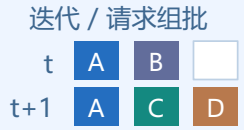
client
真实流量 / 回放
→
infer_proxy
全局调度 / 请求分发
→
Lyra 模拟节点
常驻 HTTP / 兼容接口与指标

节点内部：Lyra Runtime / 逐 iteration 仿真

机制示意

Continuous Batching

虚拟时钟 / 墙钟同步
按 token budget 动态组批
支持 Chunked Prefill



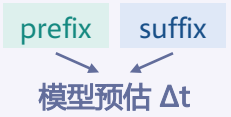
PrefixCacheTracker

前缀哈希 / 引用计数
统一块池：容量与淘汰
Full / Sparse / Linear Attention



Infertini 单步耗时预估

算子 / 模型级建模 / 异构卡
缓存 token → prefix
本步计算 token → suffix



PD 分离与 KV 传输

支持静态 / 动态 PD / 池化
各类并行 / 部署策略精细建模
KV 带宽与传输延迟建模



Hy3 线上对照

与真实线上集群对比

<1%	前缀缓存命中率偏差
<5%	TTFT 误差
<10%	TPOT 误差

Hy3-256k-amd 部署调优

线上缓存命中率提升约

1.2%

TTFT 与超时无上涨

验证闭环：理论分析 → 算法创新 → 仿真验证 → 线上优化

取舍：以 CPU 仿真换取低成本、可复现评测，代价是误差带（TTFT <5%、TPOT <10%）| 适用边界：适合策略对比与参数寻优，最终以线上压测确认

总体成效：多项优化协同后的服务能力

回到开场的服务目标：首字更快、逐字更流畅、同样资源承载更多请求，而且质量不降

大规模生产集群全量上线
混元、DeepSeek 等模型

核心算子进入 vLLM / SGLang 主线
作为原生后端随官方镜像交付

Lyra 仿真对照线上
TTFT 误差 <5% · TPOT 误差 <10%

及时 首字更快 — TTFT

-48.1%	全局调度：固定并发首字时延 (03)
3.8 ~ 4.7×	长上下文 Prefill 首字加速 (03)
-24%	Hy3（vLLM）平均首字时延 (02)

流畅 逐字更快 — TPOT

1.98 ~ 2.40×	并行投机解码，对自回归 (03)
-48.8%	SGLang 长文本逐字时延（最高） (02)
-17%	Hy3（vLLM）平均逐字时延 (02)

承载 更多请求 — 吞吐 · 并发

+44%	Decode 吞吐，并发 ×2 (03)
+39.6%	全局调度：固定并发 TPS (03)
+30%	混元生产 QPM（DeepSeek +17%） (02)

准确 质量不降 — 精度 · 分布

≤1%	4 比特量化与 BF16 基线差距 (03)
不变	投机解码严格保持输出分布 (03)
FP8	KV 卸载保持原精度，不再压 FP4 (03)

标签表示收益来源：02 算子级优化　03 系统级协同

算子让每一步更快，系统扩大容量、组织分工、减少等待：合起来才成为用户感知到的服务能力

各项为对应模型、场景与基线下的实测，口径不同，不可相加或相乘

四个认知与优化闭环

可迁移的不是某一种并行度或融合方式，而是理解目标、瓶颈、资源与协同的方法

01 目标观

优化服务，而不是优化跑分

- 质量与时延是门槛，单位资源的达标产出才是目标
- 单个 Kernel 更快，不等于用户更快：要看它是否在关键路径上

例：每一步都更快了，容量与排队仍会限制产出

02 瓶颈观

最慢的环节决定下一步

- 先量化余量，再选手段：时延有余量就换容量，显存有余量就换时延
- 突破一个瓶颈后约束会转移：容量解决后，排队成为新瓶颈

例：Decode 单步远低于时延目标 → 瓶颈在显存

03 资源观

优化是资源交换，不是免费加速

- 融合：以算子复杂度换搬运
- 并行：以通信换分工（DW 档位随长度切换）
- 卸载：以 PCIe 带宽换容量
- 压缩：付出误差预算　投机：以校验换步数

04 系统观

跨层共设计，才能兑现局部收益

- 算子决定执行效率，框架组织并行与容量，调度决定请求去哪
- 三层共同受质量约束：量化精度、输出分布都要同验

例：先用 Lyra 仿真验证，再上线压测确认

优化闭环

收益兑现后，回到新的瓶颈

1 **定义目标**
TTFT / TPOT / 吞吐 / 质量 / 成本

↓

2 **定位瓶颈**
算 / 搬 / 存 / 等：量化余量

↓

3 **提出交换**
用什么资源，换什么收益

↓

4 **验证收益**
仿真回放 → 上线压测 → 线上调优

↻

先问服务被什么限制，再问用什么资源换什么，最后看收益能否在系统中保留下来

四个认知由本次分享案例归纳，不是新增实验；各案例基线独立，收益不可累计

感谢聆听

谢谢 · Q&A

从单点加速到系统最优
大模型推理的全栈优化实践

目标与瓶颈分析 | 算子级优化 | 系统级协同 | 验证与成果

腾讯混元 AI Infra

开源与交流

将优化能力沉淀为可复用的组件

开源组件

- AngelSpec
- HPC-Ops
- AngelSlim

社区主线

HPC-Ops 进入 vLLM 与 SGLang 社区主线

Attention、MoE 作为原生后端
Router GEMM 接入 SGLang，随官方开发镜像交付

复用验证

在 Qwen3、LongCat 等模型中完成复用验证

降低开发者的适配与维护成本

技术分享

长上下文稀疏注意力精确 Top-K 技术报告

系列技术文章：算法设计、算子实现与规模化服务实践

Tencent 腾讯

感谢聆听

谢谢 · Q&A

从单点加速到系统最优
大模型推理的全栈优化实践

目标与瓶颈分析 | 算子级优化 | 系统级协同 | 验证与成果

腾讯混元 AI Infra

开源与交流

将优化能力沉淀为可复用的组件

开源组件

- AngelSpec
- HPC-Ops
- AngelSlim

社区主线

HPC-Ops 进入 vLLM 与 SGLang 社区主线

Attention、MoE 作为原生后端
Router GEMM 接入 SGLang，随官方开发镜像交付

复用验证

在 Qwen3、LongCat 等模型中完成复用验证

降低开发者的适配与维护成本

技术分享

长上下文稀疏注意力精确 Top-K 技术报告

系列技术文章：算法设计、算子实现与规模化服务实践

Tencent 腾讯

感谢聆听

谢谢 · Q&A

从单点加速到系统最优
大模型推理的全栈优化实践

目标与瓶颈分析 | 算子级优化 | 系统级协同 | 验证与成果

腾讯混元 AI Infra

开源与交流

将优化能力沉淀为可复用的组件

开源组件

- AngelSpec
- HPC-Ops
- AngelSlim

社区主线

HPC-Ops 进入 vLLM 与 SGLang 社区主线

Attention、MoE 作为原生后端
Router GEMM 接入 SGLang，随官方开发镜像交付

复用验证

在 Qwen3、LongCat 等模型中完成复用验证

降低开发者的适配与维护成本

技术分享

长上下文稀疏注意力精确 Top-K 技术报告

系列技术文章：算法设计、算子实现与规模化服务实践

Tencent 腾讯